

Тема урока: частые сценарии

1. Подсчет количества
2. Вычисление суммы и произведения
3. Обмен значений переменных
4. Сигнальные метки
5. Определение максимума и минимума
6. Расширенные операторы присваивания (`+=` , `-=` , `//=` и т.д.)

Аннотация. Рассмотрим частые сценарии при написании циклов.

Подсчет количества

Нередко нужно, чтобы наши программы подсчитывали, сколько раз что-либо произошло. К примеру, видеоигра может подсчитывать количество поворотов персонажа, или математическая программа может считать, как много чисел обладают некоторым свойством. Ключ к подсчету – использование **переменной счетчика**.

Напишем программу, которая считывает 10 чисел и определяет, сколько из них больше 10.

```
counter = 0
for _ in range(10):
    num = int(input())
    if num > 10:
        counter = counter + 1

print('Было введено', counter, 'чисел, больших 10.')
```

Каждый раз, когда мы считываем число, большее 10, мы добавляем 1 к нашему текущему значению переменной `counter`. В программе это реализовано в строке `counter = counter + 1`. Обратите внимание на начальное значение переменной счетчика `counter = 0`. Без начального значения мы получили бы ошибку, поскольку, дойдя до строки `counter = counter + 1`, Python ничего не знал бы о переменной `counter`. Строка кода `counter = counter + 1` означает: возьми старое значение переменной `counter`, прибавь к нему 1 и переприсвой переменной это значение. Если не придать переменной начальное значение, то непонятно, к чему прибавлять 1 в самый первый раз.

Подсчет количества – это очень частый сценарий. Он состоит из двух шагов:

1. Создание **переменной счетчика**, и придание ей первоначального значения:

```
counter = 0;
```

2. Увеличение переменной счетчика на 1: `counter = counter + 1`.

Часто при написании программ требуется использовать несколько счетчиков.

Модифицируем предыдущую программу: посчитаем еще и количество нулей среди введенных чисел.

```
counter1 = 0
counter2 = 0
for _ in range(10):
    num = int(input())
    if num > 10:
        counter1 = counter1 + 1
    if num == 0:
        counter2 = counter2 + 1

print('Было введено', counter1, 'чисел, больших 10.')
print('Было введено', counter2, 'нулей.' )
```

Рассмотрим еще один пример: подсчитать количество чисел из диапазона [1; 100], квадрат которых оканчивается на 4.

```
counter = 0
for i in range(1, 101):
    if i**2 % 10 == 4:
        counter = counter + 1

print(counter)
```

Мы используем функцию `range()` с двумя параметрами для генерации последовательности чисел от 1 до 100. Каждый раз, когда переменная `i` последовательно принимает значения от 1 до 100, мы проверяем условие: `i**2 % 10 == 4` (оканчивается ли квадрат числа `i` на 4).



Для переменной счетчика удобно использовать имя `counter` (или более сокращенно `cnt`).

Вычисление суммы и произведения

Наравне с подсчетом количества по частоте стоит задача вычисления суммы. К примеру, видеоигра должна считать сумму очков. В таком случае начальное значение переменной будет равно 0, а далее оно будет увеличиваться на некоторое количество заработанных очков, скажем на 10. Мы пишем следующий код:

```
score = 0
...
score = score + 10
```

Напишем программу, которая считывает 10 чисел и определяет сумму тех из них, которые больше 10.

```
total = 0
for _ in range(10):
    num = int(input())
    if num > 10:
        total = total + num

print('Сумма чисел больших 10 равна', total)
```

Каждый раз, когда программа считывает число, большее 10, она добавляет его к текущему значению переменной `total`. Это реализовано в строке `total = total + num`. Обратите внимание на начальное значение переменной-сумматора `total = 0`. Без начального значения мы получили бы ошибку, поскольку, дойдя до строки `total = total + num`, Python ничего не знал бы о переменной `total`. Строка кода `total = total + num` означает: возьми старое значение переменной `total`, прибавь к нему `num` и переприсвой переменной это значение. Если не придать переменной начальное значение, то не к чему прибавлять `num` в самый первый раз.

Подсчет суммы состоит из двух шагов:

1. Создание переменной **сумматора** и придание ей первоначального значения:
`total = 0`;
2. Увеличение переменной сумматора на нужное число: `total = total + num`.

Напишем программу, которая считает сумму натуральных чисел от 1 до 100:

```
total = 0
for i in range(1, 101):
    total = total + i

print('Сумма равна', total)
```

Рассмотрим еще один пример: напишем программу, которая запрашивает 10 целых чисел и находит их среднее значение:

```
total = 0
for _ in range(10):
    num = int(input())
    total = total + num

average = total / 10
print('Среднее значение равно', average)
```

Аналогичным образом вычисляется произведение. При вычислении произведения начальное значение переменной **мультипликатора** мы устанавливаем равным 1, в отличие от сумматора, где оно равно 0.



Для переменной сумматора и мультипликатора удобно использовать имя

`total`.

Обмен значений переменных

Очень часто нам требуется обменять значения двух переменных `x` и `y`. Начинающие программисты иногда пишут такой код:

```
x = y
y = x
```

Однако он не работает. Предположим, что `x = 3` и `y = 5`. Первая строка присвоит переменной `x` значение 5, что правильно, однако вторая строка установит значение переменной `y` в 5, поскольку значение `x` уже равно 5. Для решения задачи мы можем использовать **временную переменную**:

```
temp = x
x = y
y = temp
```

Такой код пишут почти во всех языках программирования. Однако в Python есть и более простой способ. Мы можем написать так:

```
x, y = y, x
```

В результате выполнения такого кода Python поменяет значения переменных `x` и `y` местами.

Сигнальные метки

Сигнальная метка (флажок) может использоваться, когда надо, чтобы одна часть программы узнала о происходящем в другой части программы.

Напишем программу, определяющую, что натуральное число является [простым](#):

```

num = int(input())
flag = True

for i in range(2, num):
    if num % i == 0: # если исходное число делится на какое-либо
        отличное от 1 и самого себя
        flag = False

if num == 1:
    print('Это единица, она не простая и не составная')
elif flag == True:
    print('Число простое')
else:
    print('Число составное')

```

Напомним, что число является простым, если оно не имеет делителей, кроме 1 и самого себя. Вышеприведенная программа работает следующим образом: начальное значение переменной флага равно `True`, что говорит о том, что число является простым. Затем мы перебираем все числа от 2 до `num - 1` (включительно). Если одно из этих значений оказывается делителем числа `num`, тогда число `num` является составным и мы устанавливаем значение флага `False`. Как только цикл завершен, мы проверяем, установлен флаг в `True` или нет. Если это так, мы знаем, что делитель найден не был и число является простым. В противном случае число является составным.

Флаговые переменные могут иметь более осмысленное название. Например, в случае с проверкой числа на простоту, название флаговой переменной могло бы быть `is_prime`.

Максимум и минимум

Поиск наибольшего или наименьшего значения в некоторой последовательности чисел – также частая задача в программировании. Напишем программу, которая считывает 10 **положительных чисел** и находит среди них наибольшее число.

```

largest = 0
for _ in range(10):
    num = int(input())
    if num > largest:
        largest = num

print('Наибольшее число равно', largest)

```

Мы устанавливаем начальное значение переменной `largest` в 0. Далее программа считывает 10 чисел, и если какое-то из них оказывается больше текущего значения `largest`, переприсваивает его. В качестве начального значения взято число 0, поскольку мы знаем, что все числа положительны (а 0 является максимальным неположительным числом). Таким образом, уже первое сравнение приведет к переприсваиванию.

Распространен подход, когда в качестве начального значения переменной сразу принимается первый элемент последовательности. Напишем программу, которая считывает 10 чисел (необязательно положительных) и находит среди них наибольшее:

```
largest = int(input()) # принимаем первое число за максимальное
for _ in range(9):
    num = int(input())
    if num > largest:
        largest = num

print('Наибольшее число равно', largest)
```

Для нахождения наименьшего значения последовательности следует поменять знак неравенства (`>`) на противоположный (`<`). В таком случае название переменной `largest` стоит заменить на `smallest`.



Для переменных, хранящих наибольшее и наименьшее значения, подходят имена `largest` и `smallest` соответственно.

Расширенные операторы присваивания

Довольно часто программы имеют инструкции присваивания, в которых переменная на левой стороне от оператора `=` также появляется на правой от него стороне. Например,

```
counter = counter + 1
```

На правой стороне оператора присваивания 1 прибавляется к переменной `counter`. Полученный результат затем присваивается переменной `counter`, заменяя первоначальное значение. По сути, это строка кода добавляет 1 к `counter`. Еще один пример такой инструкции мы видели при подсчете суммы:

```
total = total + num
```

Эта инструкция присваивает значение выражения `total + num` переменной `total`. В результате исполнения этой инструкции число `num` прибавляется к значению `total`.

Различные инструкции присваивания (в каждой инструкции `x = 6`)

Инструкция	Что она делает	Значение x после инструкции
<code>x = x + 4</code>	Прибавляет 4 к <code>x</code>	10
<code>x = x - 3</code>	Вычитает 3 из <code>x</code>	3
<code>x = x * 10</code>	Умножает <code>x</code> на 10	60
<code>x = x / 4</code>	Делит <code>x</code> на 4	1.5
<code>x = x // 4</code>	Делит нацело <code>x</code> на 4	1
<code>x = x % 4</code>	Находит остаток от деления <code>x</code> на 4	2

Эти типы операций находят широкое применение в программировании. Для удобства Python предлагает **расширенные операторы присваивания**. Расширенные операторы не требуют, чтобы программист дважды набирал имя переменной. Приведенную ниже инструкцию:

```
total = total + num
```

можно переписать как

```
total += num
```

Точно так же инструкцию

```
counter = counter + 1
```

можно переписать как

```
counter += 1
```

Оператор	Пример использования	Эквивалент
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 2</code>	<code>x = x - 2</code>
<code>*=</code>	<code>x *= 10</code>	<code>x = x * 10</code>
<code>/=</code>	<code>x /= 4</code>	<code>x = x / 4</code>
<code>//=</code>	<code>x //= 4</code>	<code>x = x // 4</code>
<code>%=</code>	<code>x %= 4</code>	<code>x = x % 4</code>

Тема урока: цикл `while`

1. Цикл `while`
2. Считывание данных до стоп значения
3. Бесконечный цикл
4. Решение задач

Аннотация. Урок посвящен циклу `while`. Мы научимся создавать программы, повторяющие определенные действия, пока выполняется некоторое условие.

Цикл `while`

Как уже было сказано в предыдущем уроке, существуют две основные разновидности цикла:

- циклы, повторяющиеся определенное количество раз (`for`, счетные циклы, **counting loops**);
- циклы, повторяющиеся до наступления определенного события (`while`, условные циклы, **conditional loops**)

Мы уже рассмотрели работу цикла `for`, который является счетным циклом. Цикл `for` замечательно работает, если мы заранее знаем, сколько повторений (итераций) нам потребуется сделать. Но иногда нужно, чтобы цикл выполнялся до наступления некоторого события, и количество итераций в этом случае заранее оценить просто невозможно. И здесь на помощь приходит цикл `while`.

Структура цикла `while` в Python выглядит так:

```
while условие:  
    блок кода
```

Двоеточие (`:`) в конце строки с инструкцией `while` сообщает Python, что дальше находится **блок команд**. В блок входят все строки, расположенные с отступом от строки с инструкцией `while`, вплоть до следующей строки без отступа.

Блок команд, который выполняется в цикле `while`, называется **телом цикла**.

Рассмотрим код, использующий цикл `while`, который распечатает 10 раз слово `Привет`:

```
i = 0
while i < 10:
    print('Привет')
    i += 1
```

Такой код можно легко заменить циклом `for`, поскольку мы заранее знаем, сколько раз нужно выполнить тело цикла. Однако так бывает не всегда.

Напишем программу, которая считывает числа и выводит их квадраты, пока не будет введено `-1`. При такой постановке задачи мы не можем воспользоваться циклом `for`, так как не знаем, сколько чисел будет предшествовать числу `-1`.

```
num = int(input())
while num != -1:
    print('Квадрат вашего числа равен:', num * num)
    num = int(input())
```

В качестве начального значения переменной `num`, мы используем первое из чисел. Далее, пока выполняется условие цикла, а именно пока введенное число не равно `-1`, мы исполняем тело цикла. В тело цикла входит две команды:

1. напечатать квадрат введенного числа;
2. считать следующее число.

Важным являются два момента:

1. правильная инициализация переменной `num`;
2. изменение переменной `num` внутри цикла `while`.



Важно: если не изменять переменную `num` внутри цикла, то можно получить так называемый **бесконечный цикл**, который будет выполняться бесконечно много раз.

Цикл `while` очень похож на условный оператор `if`. Разница заключается в том, что в случае с условным оператором соответствующий блок кода будет выполняться только один раз, тогда как с циклом `while` блок кода будет выполнен многократно.

Цикл for VS цикл while

Мы всегда можем заменить цикл `for` с помощью цикла `while`. Следующие две программы выводят числа от 0 до 100:

```
# используем for
for i in range(101):
    print(i)

# используем while
i = 0
while i < 101:
    print(i)
    i += 1
```

В первом цикле переменная `i` последовательно принимает значения от 0 до 100. Для цикла `while` нам пришлось завести самостоятельно переменную `i` и придать ей начальное значение. Тело цикла `while` содержит аналогичную инструкцию вывода `print(i)`, однако помимо этого мы самостоятельно увеличиваем значение переменной `i` на 1, что делается автоматически в случае с циклом `for`.

Напишем программу, выводящую все числа, кратные 3, используя цикл `for` и `while`:

```
# используем for
for i in range(0, 100, 3):
    print(i)

# используем while
i = 0
while i < 100:
    print(i)
    i += 3
```

Не всегда, однако, удастся заменить цикл `while` с помощью цикла `for`. Если заранее не известно количество итераций, то необходимо использовать цикл `while` и только его.



Считывание данных до стоп значения

Часто при решении задач на цикл `while` мы считываем данные до тех пор, пока пользователь не введет некоторое значение, которое называют **стоп значением**. Напишем программу, которая считывает числа и находит их сумму до тех пор, пока пользователь не введет слово `stop`:

```
text = input()
total = 0
while text != 'stop':
    total += int(text)
    text = input()

print('Сумма чисел равна', total)
```

Такой код будет часто использоваться при решении задач.

Бесконечный цикл

Всегда кроме редких случаев цикл `while` должен содержать возможность завершиться. То есть в цикле что-то должно сделать проверяемое условие ложным. Если цикл не имеет возможности завершиться, то он называется **бесконечным циклом**. Бесконечный цикл продолжает повторяться до тех пор, пока программа не будет прервана. Бесконечные циклы обычно появляются, когда программист забывает написать программный код внутри цикла, который делает проверяемое условие ложным. В большинстве случаев следует избегать применения бесконечных циклов.

Пример бесконечного цикла:

```
i = 0
total = 0
while i < 10:
    total += i
```

Так как в теле цикла не происходит изменения переменной `i`, то условие `i < 10` остается истинным и цикл выполняется бесконечно много раз.

Бесконечные циклы можно использовать в связке с оператором прерывания `break`. Об этом будет рассказано в следующих уроках.

Примечания

Примечание 1. Цикл `while` получил свое название из-за характера своей работы: он выполняет некую задачу до тех пор, **пока** условие является истинным. Слово *while* на английском означает как раз "пока".

Примечание 2. Цикл `while` называют циклом с **предусловием**, поскольку выполнению тела цикла предшествует проверка условия (сначала проверяется условие, а уже затем выполняется тело цикла).

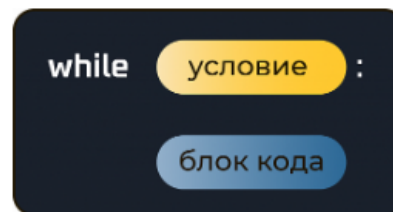
Примечание 3. Однократное выполнение тела цикла называется **итерацией** цикла.

Примечание 4. Цикл `while` может не выполниться ни одного раза. Например, следующий код:

```
i = -1
while i > 0:
    print('Hello world!')
```

не выведет текст, поскольку условие `i > 0` ложно с самого начала.

Примечание 5. Графическое представление цикла `while` имеет вид:



Примечание 6. Условие в цикле `while`, как и в условном операторе `if`, может содержать логические операции `or`, `and`, `not`.

Обработка цифр числа

При изучении целых чисел (тип данных `int`), мы говорили про операцию целочисленного деления `//` и операцию нахождения остатка от деления одного целого числа на другое `%`. Используя цикл `while` и две данных операции, можно обработать цифры числа с произвольным количеством разрядов (цифр).

Пусть дано натуральное число `n`. Тогда:

- результатом операции `n % 10` – является последняя цифра числа;
- результатом операции `n // 10` – является число с удаленной последней цифрой.

Напишем программу, которая считывает натуральное число (целое положительное) и обрабатывает его цифры.

```
n = int(input())
while n != 0: # пока в числе есть цифры
    last_digit = n % 10 # получить последнюю цифру
    # код обработки последней цифры
    n = n // 10 # удалить последнюю цифру из числа
```

Цикл `while` работает до тех пор, пока в числе есть необработанные цифры. Тело цикла содержит:

1. процедуру получения последней цифры `last_digit = n % 10`;
2. код обработки последней цифры;
3. процедуру удаления последней цифры из числа `n = n // 10`.

В качестве процедуры обработки может быть все что угодно: вывод цифр, нахождение суммы, произведения цифр, нахождение наибольшей или наименьшей цифры, подсчет цифр удовлетворяющих некоторому условию и т.д.

Напишем программу, которая определяет, есть ли в числе цифра `7`.

```
num = int(input())
has_seven = False # сигнальная метка

while num != 0:
    last_digit = num % 10
    if last_digit == 7:
        has_seven = True
    num = num // 10

if has_seven == True:
    print('YES')
else:
    print('NO')
```

Оператор прерывания цикла `break`

Иногда бывает нужно **прервать выполнение цикла преждевременно**.

Оператор `break` прерывает ближайший цикл `for` или `while`.

Усовершенствуем с помощью оператора `break` программу, проверяющую число на простоту:

```
num = int(input())
flag = True

for i in range(2, num):
    if num % i == 0:      # если исходное число делится на какое-либо
        # отличное от 1 и самого себя
        flag = False
        break            # останавливаем цикл если встретили делитель
                           # числа

if flag: # эквивалентно if flag == True:
    print('Число простое')
else:
    print('Число составное')
```

Как только мы встречаем делитель отличный от 1 и `num`, мы меняем значение сигнальной метки и прерываем цикл, поскольку дальнейшее его выполнение лишено смысла: число гарантированно не является простым.



Оператор прерывания цикла `break` позволяет ускорять программы, так как мы избавляемся от лишних итераций.

Напишем программу, использующую цикл `for`, которая считывает 10 чисел и суммирует их до тех пор, пока не обнаружит отрицательное число. В этом случае выполнение цикла прерывается командой `break`:

```
result = 0
for i in range(10):
    num = int(input())
    if num < 0:
        break
    result += num
print(result)
```



Оператор прерывания цикла `break` удобен в связке с **сигнальными метками**: когда после проверки некоторого условия нам нет смысла продолжать выполнение цикла.

Напишем программу, которая определяет, содержит ли введенное пользователем число цифру 7.

```
num = int(input())
number = num
flag = False
while num != 0:
    last_digit = num % 10
    if last_digit == 7:
        flag = True
        break          # прерываем цикл, так как число гарантированно
                        # содержит цифру 7
    num //= 10

if flag: # эквивалентно if flag == True:
    print('Число', number, 'содержит цифру 7')
else:
    print('Число', number, 'не содержит цифру 7')
```

Как только мы встретили цифру 7, мы меняем значение сигнальной метки и прерываем цикл с помощью оператора `break`. Мы можем и не прерывать цикл преждевременно, а дождаться его естественного завершения (условие `num != 0`, то есть все цифры числа обработаны), однако в таком случае мы будем совершать лишнюю работу, и в случае, если число очень большое, программа будет работать медленнее.

Бесконечные циклы

В предыдущих уроках мы говорили о цикле, который не имеет возможности завершиться и назвали его **бесконечным циклом**. Самый простой способ создать бесконечный цикл в Python – записать следующий код:

```
while True:
    print('Python is awesome!')
```

Результатом выполнения такого кода будет бесконечное количество строчек текста:

```
Python is awesome!
Python is awesome!
.
.
.
Python is awesome!
Python is awesome!
Python is awesome!
```

Бесконечный цикл продолжает повторяться до тех пор, пока программа не будет прервана. Изучив оператор `break`, мы получили механизм прерывания бесконечных циклов.

Возможно, вам может показаться, что бесконечные циклы лишены смысла, однако это не совсем так. Например, вы можете написать программу, которая запускается и работает, постоянно принимая запросы на обслуживание. Программный код такой программы может выглядеть так:

```
while True:
    query = get_new_query() # получаем новый запрос на обработку
    query.process()         # обрабатываем запрос
```

Иногда с помощью бесконечного цикла удастся сделать программный код более читабельным. Более простым может быть завершение цикла на основе условий внутри тела цикла, а не на основе условий в его заголовке:

```
while True:
    if условие 1: # условие для остановки цикла
        break
    ...
    if условие 2: # еще одно условие для остановки цикла
        break
    ...
    if условие 3: # еще одно условие для остановки цикла
        break
```

В подобных случаях, когда существует множество причин завершения цикла, часто их проще выделить из нескольких разных мест, чем пытаться указать все условия завершения в заголовке цикла.



Важно: бесконечные циклы могут быть очень полезными. Просто помните, что вы должны убедиться, что цикл в какой-то момент будет прерван, чтобы он действительно не становился бесконечным.

Оператор `continue`

Другая стандартная идиома циклов — пропуск отдельных элементов при переборе. Оператор `continue` позволяет перейти к следующей итерации цикла `for` или `while` до завершения всех команд в теле цикла.

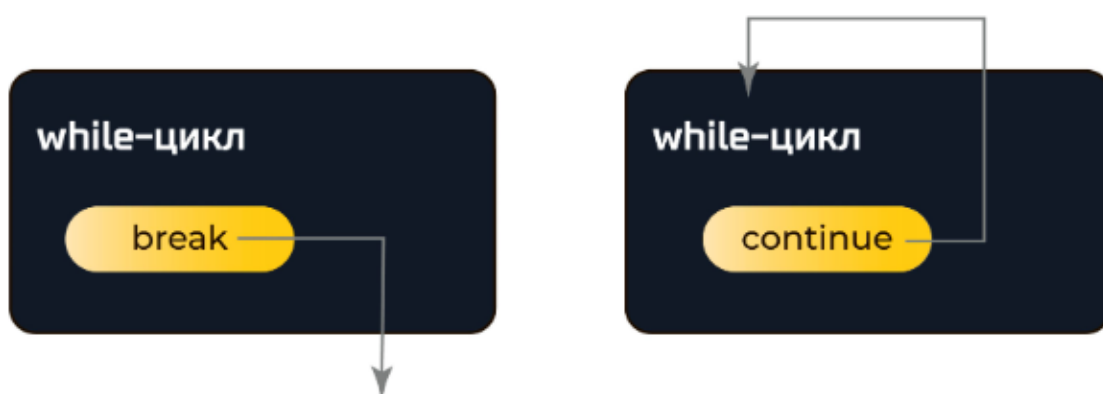
Напишем программу, которая выводит все числа от 1 до 100, кроме чисел 7, 17, 29 и 78.

```
for i in range(1, 101):
    if i == 7 or i == 17 or i == 29 or i == 78:
        continue # переходим на следующую итерацию
    print(i)
```

Примечания

Примечание 1. Оператор `break` прерывает выполнение ближайшего цикла, а не программы, то есть далее будет выполнена команда, следующая сразу за циклом.

Примечание 2. Графическое представление операторов `break` и `continue` имеет вид:



Блок else в циклах

Python допускает необязательный блок `else` в конце циклов `while` и `for`. Это уникальная особенность Python, не встречающаяся в большинстве других языков программирования. Синтаксис такой конструкции следующий:

```
while условие:
    блок кода1
else:
    блок кода2

# или

for i in range(10):
    блок кода1
else:
    блок кода2
```

Блок кода2, указанный в `else`, будет выполнен, когда **штатным образом** завершается цикл `while` или `for`.

Сейчас вы можете подумать: «Как это может быть полезным?» Ведь мы можем сделать то же самое, поместив блок кода2 сразу после цикла `while` или `for` без `else`:

```
while условие:
    блок кода1
    блок кода2

# или

for i in range(10):
    блок кода1
    блок кода2
```

В чём разница?



Если слово `else` отсутствует в описании цикла, то блок кода2 будет выполняться после завершения цикла, несмотря ни на что. Если же слово `else` присутствует, то блок кода2 будет выполняться только в том случае, если цикл завершается

штатным образом. Под **штатным завершением** цикла подразумевается его завершение **без** использования оператора прерывания `break`.

Рассмотрим следующий программный код:

```
n = 5
while n > 0:
    n -= 1
    print(n)
else:
    print('Цикл завершен.')
```

Данный цикл повторяется до тех пор, пока истинно условие `n > 0`. Поскольку цикл завершился **штатным образом**, то блок кода в инструкции `else` будет выполнен.

Таким образом, результатом выполнения такой программы будут строки:

```
4
3
2
1
0
Цикл завершен.
```

Рассмотрим следующий программный код:

```
n = 5
while n > 0:
    n -= 1
    print(n)
    if n == 2:
        break
else:
    print('Цикл завершен.')
```

Этот цикл преждевременно завершается с помощью оператора прерывания `break`, поэтому блок кода в инструкции `else` не будет выполнен. Результатом выполнения такой программы будут строки:

```
4
3
2
```



Вам может показаться, что инструкция `else` в циклах `while` и `for` не совсем соответствует тому, что реально происходит. Гвидо ван Россум, создатель Python, сказал, что если бы он проектировал язык Python заново, то избавился бы от `else` в циклах.

Напишем программу, которая определяет, содержит ли введенное пользователем число цифру 7. Вместо программного кода, написанного ранее:

```

num = int(input())
n = num
flag = False
while n != 0:
    last = n % 10
    if last == 7:
        flag = True
        break
    # прерываем цикл, так как число гарантированно
    содержит цифру 7
    n //= 10

if flag == True:
    print('Число', num, 'содержит цифру 7')
else:
    print('Число', num, 'не содержит цифру 7')

```

мы можем использовать:

```

num = int(input())
n = num
while n != 0:
    last = n % 10
    if last == 7:
        print('Число', num, 'содержит цифру 7')
        break
    n //= 10
else:
    print('Число', num, 'не содержит цифру 7')

```

Примечания

Примечание 1. Оператор `continue` не влияет на выполнение блока `else` в циклах.

Примечание 2. Блок `else` в циклах часто применяется для обработки отсутствия элементов.

Примечание 3. Блок кода `else` в циклах встречается не так часто на практике. Однако если вы обнаружите ситуацию, в которой применение `else` оправдано, то не стесняйтесь его использовать. Это может добавить ясности вашему коду!

Тема урока: вложенные циклы

1. Вложенные циклы
2. Операторы `break` и `continue` во вложенных циклах
3. Решение задач

Аннотация. Вложенные, находящиеся внутри других циклов, циклы.

Вложенные циклы

Вложенный цикл расположен в еще одном цикле. Часы служат хорошим примером работы вложенного цикла. Секундная, минутная и часовая стрелки вращаются вокруг циферблата. Часовая стрелка смещается всего на 1 шаг для каждых 60 шагов минутной стрелки. И секундная стрелка должна сделать 60 шагов для 1 шага минутной стрелки. Это означает, что для каждого полного оборота часовой стрелки (12 шагов), минутная стрелка делает 720 шагов.



Рассмотрим цикл, который частично моделирует электронные часы. Он показывает секунды от 0 до 59:

```
for seconds in range(60):  
    print(seconds)
```

Можно добавить переменную `minutes` и вложить цикл написанный выше внутрь еще одного цикла, который повторяется 60 раз:

```
for minutes in range(60):  
    for seconds in range(60):  
        print(minutes, ': ', seconds)
```

Для того, чтобы сделать модель законченной, можно добавить еще одну переменную для подсчета часов:

```
for hours in range(24):  
    for minutes in range(60):  
        for seconds in range(60):  
            print(hours, ': ', minutes, ': ', seconds)
```

Результатом работы такого кода будет:

```
0 : 0 : 0  
0 : 0 : 1  
0 : 0 : 2  
...  
23 : 59 : 58  
23 : 59 : 59
```

Самый внутренний цикл сделает 60 итераций для каждой итерации среднего цикла. Средний цикл сделает 60 итераций для каждой итерации самого внешнего цикла. Когда самый внешний цикл сделает 24 итерации, средний сделает $24 \cdot 60 = 1440$ итераций, и самый внутренний цикл сделает $24 \cdot 60 \cdot 60 = 86400$ итераций!

Пример имитационной модели часов подводит нас к нескольким моментам, имеющим отношение к вложенным циклам:

- вложенный цикл выполняет все свои итерации для каждой отдельной итерации внешнего цикла;
- вложенные циклы завершают свои итерации быстрее, чем внешние циклы;
- для того, чтобы получить общее количество итераций вложенного цикла, надо **перемножить** количество итераций всех циклов.



Мы можем вкладывать друг в друга циклы как `for`, так и `while`.

Операторы `break` и `continue` во вложенных циклах

Оператор `break` выполняет прерывание **ближайшего** цикла в котором он расположен. Аналогично, оператор `continue` осуществляет переход на следующую итерацию **ближайшего** цикла.

Рассмотрим программный код:

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

Результатом его выполнения будут 9 строк:

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

Изменим код, добавив во вложенный цикл условный оператор с оператором `break`:

```
for i in range(3):
    for j in range(3):
        if i == j:
            break
        print(i, j)
```

Результатом выполнения нового кода будут 3 строки:

```
1 0
2 0
2 1
```

Изменим оператор прерывания `break` на оператор `continue`:

```
for i in range(3):
    for j in range(3):
        if i == j:
            continue
        print(i, j)
```

Результатом выполнения нового кода будут 6 строк:

```
0 1
0 2
1 0
1 2
2 0
2 1
```



Если необходимо добиться прерывания внешнего цикла из-за выполнения условия во внутреннем, то стоит сделать это через **сигнальную метку**.

Примеры решения задач

Один интересный способ узнать о работе вложенных циклов состоит в их использовании для вывода на экран комбинаций символов. Давайте взглянем на один простой пример. Предположим, что мы хотим напечатать на экране звездочки в виде прямоугольной таблицы:

```
*****
*****
*****
*****
*****
*****
*****
*****
```

Таблица состоящая из звездочек состоит из 8 строк и 6 столбцов. Приведенный ниже фрагмент кода можно использовать для вывода одной строки звездочек:

```
for i in range(6):
    print ( '*', end='')
```

Для того чтобы завершить весь вывод таблицы звездочек, нам нужно выполнить этот цикл восемь раз. Мы можем поместить этот цикл в еще один цикл, который делает восемь итераций, как показано ниже:

```
for i in range(8):
    for j in range(6):
        print('*', end='')
    print()
```

Внешний цикл делает восемь итераций. Во время каждой итерации этого цикла внутренний цикл делает 6 итераций. (Обратите внимание, что в строке 4 после того, как все строки были напечатаны, мы вызываем функцию `print()`. Мы должны это сделать, чтобы в конце каждой строки продвинуть экранный курсор на следующую строку. Без этой инструкции все звездочки будут напечатаны на экране в виде одной длинной строки.)



Давайте рассмотрим еще один пример. Предположим, что вы хотите напечатать звездочки в комбинации, которая похожа на приведенный ниже звездный треугольник:

```
*  
**  
***  
****  
*****  
*****  
*****  
*****  
*****
```

Представим эту комбинацию звездочек, как сочетание строк и столбцов. В этой комбинации всего восемь строк. В первой строке один столбец. Во второй строке – два столбца. В третьей строке – три. И так продолжается до восьмой строки, в которой восемь столбцов.

```
for i in range(8):  
    for j in range(i + 1):  
        print('*', end='')  
    print()
```